快速开始 · 指令集

> 适用板卡：**MLK-AFC03**（RK3576J + 安路 PH1A90SEG324 异构核心板）
> 链路： `IMX415 →(MIPI x4)→ FPGA →(MIPI x4)→ RK3576 → HDMI / 网页 MJPEG`
> 本文只列**当前已实现功能**的启动/操作指令，一行一条，可直接复制。
> 详细原理见 `README.md` 、 `rk/web/README.md` 、 `rk/deploy/README.md` 、 `rk/ai/README.md` 。

0. 前提与约定

项	值
板子	`ssh afc03` （= `root@192.168.137.100` ，已配免密）
PC 工作目录	`08_csi2rk_mipi_imx415/`
板上目录	`/root/rk/{web,deploy,driver,dts,ai}` 、云台脚本 `/root/gimbal_*.py`
FPGA bit	`fpga/fpga_prj_compress.bit` （含 I2C 从机 0x40 + 预处理 + cam_param）
RK 内核	`rk/boot.img` （内置 `milianke_mipi` 驱动 + BT.709/limited 色彩补丁）
采集节点	`/dev/video22` （rkcif，Multiplanar，NV16 1920x1080）
板内控制口	`I2C9 @ 0x40` （RK↔FPGA 单条 I2C 反向传参通路，底板已连好，无需飞线）

上电顺序：**先烧 FPGA bit → RK 上电 → 在 RK 上开流**（ `I_dsi_en` 由 RK 驱动拉高，FPGA 才会发数）。

1. 当前已实现的功能

#	功能	状态	入口 / 指令
1	链路自检（不依赖显示器）	✅	`rk/deploy/rk_mipi_selftest.sh`
2	HDMI 直接出图 **53.7 fps**	✅	`rk/deploy/test_hdmi.sh`

#	功能	状态	入口 / 指令
3	网页 MJPEG 推流 **1920x1080@59.6fps**	✅	`rk/web/start_webstream.sh` （:9012）
4	软件画面调参（亮度/对比度/饱和度/色相/伽马/镜像/翻转/旋转/缩放/JPEG 质量/帧率上限）	✅	网页「画面」页签 ↔ `/api/params`
5	**IMX415 摄像头参数**（曝光 0.05~16ms / 增益 0~30dB(0.3dB/step) / EV -3~+3 / 白平衡 R、B / ROI）	✅	网页「摄像头」页签 ↔ `/api/fpga`
6	**FPGA 预处理**（初级多选：灰度/高斯/直方图均衡/对比度/反色；高级单选：Sobel/阈值/形态/中值/拉普拉斯/浮雕/Prewitt/锐化）	✅	网页「预处理」页签 ↔ `/api/fpga`
7	输出格式切换 **1080P@60 / 4K@25 / 4K@30**（同时写 FPGA + 重设 media pipeline）	✅ （4K@30 带宽 99.5% 实测跑不动）	网页「摄像头」页签下拉框 ↔ `/api/mode`
8	720P@90	❌ FPGA RTL 未实现	页面标注并拦截
9	**AI 人体检测 → 17 关节姿态 → 8 类危险动作**（RK3576 NPU）	✅	`AI=1 ./start_webstream.sh` （集成进 9012）
10	参数自动存档（重启/换浏览器自动恢复）	✅	服务端 `state.json`
11	云台 QGimbal 键盘控制（含回零/设零点）	✅	`/root/gimbal_keyctrl.py`
12	链路比特精确性验证 / 颜色诊断 / 性能基准 / 环境探测	✅	`deploy/` 、 `web/` 下的诊断脚本

2. 一次性部署（PC → 板）

```
# 2.1 拷贝所有运行时代码到板子（已配免密，直接 scp 到别名）
scp -r rk/web rk/deploy rk/ai afc03:/root/rk/
ssh afc03 "chmod +x /root/rk/web/*.sh /root/rk/deploy/*.sh /root/rk/ai/board/*.sh"

# 2.2 云台脚本（固定在板子 /root/ 下）
scp rk/gimbal/gimbal_keyctrl.py rk/gimbal/gimbal_cmd.py afc03:/root/
ssh afc03 "chmod +x /root/gimbal_*.py"

# 2.3 首次刷 boot 分区（内核 + dtb + 驱动；只需一次）
scp rk/boot.img afc03:/root/boot.img
ssh afc03 "/root/rk/deploy/flash_boot.sh /root/boot.img --yes"
```

3. 日常指令集

3.1 三件最常用的事

```
# ① 链路自检（先跑这个，确认 FPGA→MIPI→RK 数据通路是通的）
ssh afc03 "cd /root/rk/deploy && ./rk_mipi_selftest.sh"

# ② 网页推流（PC 浏览器开 mipi_viewer.html，地址填 192.168.137.100:9012）
ssh afc03 "cd /root/rk/web && ./start_webstream.sh"

# ③ HDMI 全屏出图
ssh afc03 "cd /root/rk/deploy && ./test_hdmi.sh"
```

3.2 网页推流 9012

```
# 默认：/dev/video22 → :9012，1080p60
ssh afc03 "cd /root/rk/web && ./start_webstream.sh"

# 指定设备/端口 + 降压（低带宽或 CPU 紧张时）
ssh afc03 "cd /root/rk/web && ./start_webstream.sh /dev/video22 9012 --stream-width 960 --qualit

# 不依赖摄像头/FPGA 的彩条测试图案（验证网页链路）
ssh afc03 "cd /root/rk/web && ./start_webstream.sh x 9012 --backend test"

# 强制走 GStreamer 原生管线（系统 python3，CPU 更低，但画质不展开 16-235）
ssh afc03 "cd /root/rk/web && PYTHON=python3 ./start_webstream.sh /dev/video22 9012 --backend gs

# 改完 py/html 后重启服务并自检接口
ssh afc03 "cd /root/rk/deploy && ./restart_web.sh"
```

PC 侧观看：双击 `rk/web/mipi_viewer.html` ，地址框填 `192.168.137.100:9012` ；
或直接开 `http://192.168.137.100:9012/` ；VLC 可放 `http://192.168.137.100:9012/stream` 。

手机 / 平板（无线局域网）：板子 USB WiFi 由 NetworkManager 管理，连 `509` （5GHz），开机自动 DHCP，当前地址 `http://192.168.1.136:9012/` 。页头「局域网」芯片会显示 `/api/net` 选出的无线地址，点一下即可复制。排障：

```
ssh afc03 "nmcli -t dev status | grep wlx"                  # 应为 wifi:connected:509
ssh afc03 "ip -4 -o addr show wlx8c7ab3f8171b"              # 应有 192.168.1.x
ssh afc03 "nmcli con up 509"                                # 掉线时手动重连
```

> ⚠ **不要**再通过 `/etc/wpa_supplicant.conf` 手工起 `wpa_supplicant` / `dhclient`（曾写在 `load_drv.sh` 里）：会和 NetworkManager 抢同一张网卡，表现为网卡 `associated` 但没有 IP，局域网/手机完全打不开。无线统一交给 NetworkManager。

> ⚠ 启动脚本**没有 `--fps` 参数**（`mipi_webstream.py` 未定义它，传了会直接报错退出）。推流帧率上限在**网页「画面」页签的「帧率」滑块**上调，或 `POST /api/params {"fps":20}` 。采集尺寸/JPEG 质量则在启动时用 `--width/--height` / `--quality` 定。

3.3 HDMI 出图 / 抓帧

```
ssh afc03 "cd /root/rk/deploy && ./hdmi_view.sh"              # 高帧率版（53.7fps，自动缩放到屏幕
ssh afc03 "cd /root/rk/deploy && ./test_hdmi.sh /dev/video22" # 标准版，全屏
ssh afc03 "cd /root/rk/deploy && ./test_hdmi.sh /dev/video22 save"   # 抓一帧存 /root/frame.png
ssh afc03 "gst-launch-1.0 v4l2src device=/dev/video22 ! kmssink"     # 官方原始等价命令
```

3.4 摄像头参数（曝光 / 增益 / EV / 白平衡 / ROI）

网页「摄像头」页签拖滑块即可；命令行等价写法（走 `/api/fpga`，服务需在跑）：

```
# 读当前值
ssh afc03 "curl -s http://127.0.0.1:9012/api/fpga"

# 设曝光 8ms + 增益 9dB + EV +1 + 白平衡 R120%/B90% + ROI 80%
ssh afc03 "curl -s -X POST http://127.0.0.1:9012/api/fpga -H 'Content-Type: application/json' \
  -d '{\"cam_exp_ms\":8,\"cam_gain_db\":9,\"ev\":1,\"wb_r_pct\":120,\"wb_b_pct\":90,\"roi_pct\"

# 5 项全验收（曝光 A/B、增益阶梯、EV、稳定性、连续改参 20 次压力）
ssh afc03 "/opt/miniforge3/envs/mlk-ov/bin/python /root/rk/deploy/verify_cam_final.py"

# 看参数有没有真的写进 FPGA（CAM_WR_CNT 应随每次改参增长）
ssh afc03 "python3 /root/rk/deploy/raw_cam_dump.py"
```

> 物理量：曝光 `cam_exp_ms` 0.05~16ms、增益 `cam_gain_db` 0~30dB、 `ev` -3~+3、 `wb_r_pct` / `wb_b_pct` 白平衡增益（`WB_R/WB_B` 寄存器 128 = 1.0x）、 `roi_pct` ROI 25~100%（100 = 全画幅）。
> 参数寄存器： `0xC0~0xC2` =CAM_EXP(L/M/H, 20 位)、 `0xC3` =CAM_GAIN、 `0xC4` =EV_BIAS(8=0EV)、 `0xC5/0xC6` =WB_R/WB_B、 `0xC7` =FPS_DIV、 `0xC8` =ROI_PCT、 `0xC9` =CAM_TRIG(写任意值触发一次下发)；
> 状态： `0xD0` =CAM_WR_CNT(写成功次数)、 `0xD2` =CAM_BUSY、 `0xD3` =CAM_ID(0xA5 表示参数区存在)；
> 分辨率： `0xE0` =RES_MODE（0=1080p60 / 1=4K30 / 2=4K25）。

3.5 输出格式切换（分辨率 / 帧率）

网页「摄像头」页签 →「输出格式」下拉框（切换时画面断 1~2 秒）。命令行：

```
Y=/opt/miniforge3/envs/mlk-ov/bin/python

# 只查看当前状态（不动任何东西）
ssh afc03 "cd /root/rk/deploy && ./switch_res_mode.sh status"

# 切 1080P@60（默认，载荷 49.8%，最稳）
ssh afc03 "cd /root/rk/deploy && ./switch_res_mode.sh 1080p"

# 切 4K@25（载荷 82.9%，实测可反复切）
ssh afc03 "cd /root/rk/deploy && ./switch_res_mode.sh 4k25"

# 切 4K@30（载荷 99.5% —— 超带宽，实测跑不动，一般不要用）
ssh afc03 "cd /root/rk/deploy && ./switch_res_mode.sh 4k30"

# 经网页接口切（等价；id 见 GET /api/modes）
ssh afc03 "curl -s -X POST http://127.0.0.1:9012/api/mode -H 'Content-Type: application/json' -d
```

> ⚠ 4K 必须同时满足：FPGA `RES_MODE` + RK 侧 media pipeline（必须从 **sensor subdev** 用 `media-ctl` 设起，
> 只改 `/dev/video22` 会被 rkcif 拉回）。 `switch_res_mode.sh` 已封装好这两步。
> 4K 排障用 `ssh afc03 "cd /root/rk/deploy && ./diag_4k25.sh"` 。

3.6 FPGA 预处理（初级多选 + 高级单选）

网页「预处理」页签；命令行：

```
# 板端工具（直写 I2C，不需要 web 服务在跑）
ssh afc03 "python3 /root/rk/deploy/fpga_cfg.py dump"        # 打印全部寄存器
ssh afc03 "python3 /root/rk/deploy/fpga_cfg.py heartbeat"   # 连读心跳，验证 I2C 通路活着

# 初级（多选位图 0xF0）：bit0 灰度 bit1 高斯 bit2 直方图均衡 bit3 对比度 bit4 反色
ssh afc03 "python3 /root/rk/deploy/fpga_cfg.py set 0xF0 1"  # 只开灰度
ssh afc03 "python3 /root/rk/deploy/fpga_cfg.py set 0xF0 6"  # 高斯 + 直方图均衡
ssh afc03 "python3 /root/rk/deploy/fpga_cfg.py set 0xF0 31" # 五项全开

# 高级（单选 0xB1）：0=关 1=Sobel 2=阈值 3=形态 4=中值 5=拉普拉斯 6=浮雕 7=Prewitt 8=锐化
ssh afc03 "python3 /root/rk/deploy/fpga_cfg.py set 0xB0 1"
ssh afc03 "python3 /root/rk/deploy/fpga_cfg.py set 0xB1 4"  # 中值
ssh afc03 "python3 /root/rk/deploy/fpga_cfg.py set 0xB3 100" # 阈值分割阈值

# 经网页接口（服务在跑时）
ssh afc03 "curl -s -X POST http://127.0.0.1:9012/api/fpga -H 'Content-Type: application/json' \
  -d '{\"PP_EN\":1,\"ADV_MODE\":1,\"ADV_SOBEL_SH\":2}'"
ssh afc03 "curl -s -X POST http://127.0.0.1:9012/api/fpga -H 'Content-Type: application/json' \
  -d '{\"BASIC_EN\":6,\"BASIC_GAUSS\":8,\"BASIC_HE\":255}'"
```

> **寄存器**
>
> - 初级： `0xF0` =BASIC_EN（5 位位图）、 `0xF1` =灰度系数档（0=601/1=709）、 `0xF2` =高斯强度、
>   `0xF3` =直方图均衡强度、 `0xF4` =对比度强度
> - 高级： `0xB0` =PP_EN、 `0xB1` =ADV_MODE（0..8）、 `0xB2` =Sobel 位移、 `0xB3` =阈值、
>   `0xB4` =0 腐蚀/1 膨胀、 `0xB5` =形态学阈值、 `0xB6` =拉普拉斯位移、 `0xB7` =Prewitt 位移、
>   `0xF5` =浮雕增益、 `0xF6` =浮雕偏置、 `0xF7` =锐化强度
> - 状态： `0xB8~0xBB` =帧计数、 `0xBC` =心跳（只读）、 `0xA9/0xAA` =版本号（V45.2= `0x4502` ）
>
> 初级固定串联顺序：**高斯 → 直方图均衡 → 灰度 → 对比度 → 反色**；两级可同时生效。
> 注意：**打开任意一级预处理会自动把 AI 画面源切回原图**（双向联动）。

3.7 AI 人体检测 / 姿态 / 危险动作（NPU）

```
# 推荐：直接集成进 9012（复用已转好的 BGR 帧，无第二路流、无二次编解码）
ssh afc03 "cd /root/rk/web && AI=1 ./start_webstream.sh"            # 默认画面源=原图
ssh afc03 "cd /root/rk/web && AI=1 AI_SRC=ai ./start_webstream.sh" # 启动即 AI 标注画面
ssh afc03 "cd /root/rk/web && AI=1 AI_FPS=8 AI_INTERVAL=2 ./start_webstream.sh"
ssh afc03 "cd /root/rk/web && AI=1 AI_ZONE='0.2,0.6;0.8,0.6;0.8,1;0.2,1' ./start_webstream.sh"

# 看 AI 状态（fps / fps_det / det_ms / pose_ms / persons / detections / events）
ssh afc03 "curl -s http://127.0.0.1:9012/api/ai"

# 在线调 AI（切画面源/开关/帧率/阈值/灵敏度）
ssh afc03 "curl -s -X POST http://127.0.0.1:9012/api/ai -H 'Content-Type: application/json' \
  -d '{\"source\":\"ai\",\"ai_fps\":10,\"det_conf\":0.4,\"tune\":{\"fall\":1.0,\"run\":1.0}}'"

# 独立版服务（9013）：需要从别处取流（RTSP / 图片 / 别的 MJPEG）时才用
ssh afc03 "SRC=http://127.0.0.1:9012/stream PORT=9013 AIFPS=6 bash /root/rk/ai/board/start_ai.sh
ssh afc03 "SRC=/dev/video22 PORT=9013 bash /root/rk/ai/board/start_ai.sh"
```

网页顶栏「画面源」在**原图 / AI 标注**间随时切换；「AI 标注」页签可切模型档位、AI 帧率、阈值、关节范围、跌倒/奔跑灵敏度、纯骨骼（黑底）隐私模式。实测：视频仍 59.6fps，YOLO 26fps，单进程 CPU 270%。

3.8 云台（QGimbal，USB CDC-ACM `/dev/ttyACM0`）

```
# 键盘方向键控制（↔ Yaw、↑↓ Pitch、空格停、双击空格回零、Ctrl+空格设零点、q 退出）
ssh -t afc03 "/root/gimbal_keyctrl.py --port /dev/ttyACM0"

# 直接发固件命令并原样打印回显（列配置 / 看状态 / 看诊断）
ssh afc03 "/root/gimbal_cmd.py --port /dev/ttyACM0 'config --list'"
ssh afc03 "/root/gimbal_cmd.py --port /dev/ttyACM0 'status'"
ssh afc03 "/root/gimbal_cmd.py --port /dev/ttyACM0 'diag'"

# 往返老化测试（pitch ±0.9rad 往返）
ssh afc03 "/usr/gimbal/usb_cycle.py --port /dev/ttyACM0"
ssh afc03 "pkill -INT -f usb_cycle.py"     # 安全停止（会打印 Both axes disabled.）
```

> 串口是独占的，跑之前先关掉其它串口客户端/网页上位机。
> 若终端按住方向键出现「速度先降后升」，把 `--release-ms` 调到 1000。

3.9 诊断 / 验证

```
# 环境探测（逐个解释器检查 cv2/numpy/PIL/gi、GStreamer 元素、采集节点与格式）
ssh afc03 "cd /root/rk/deploy && ./env_check.sh"

# 颜色诊断（U/V 顺序 × 601/709 × limited/full 全渲染对比，终端打印 Y/Cb/Cr 统计）
ssh afc03 "pkill -f 'mipi_web[s]tream'; /opt/miniforge3/envs/mlk-ov/bin/python /root/rk/web/colo

# 链路比特精确性验证（先让出 /dev/video22 再抓原始帧比对）
ssh afc03 "sh /root/rk/deploy/run_verify.sh"          # 默认 /dev/video22 抓 3 帧
ssh afc03 "sh /root/rk/deploy/run_verify.sh /dev/video22 10"

# 帧率基准（各阶段耗时 / CPU 亲和性 / 显示 vs 采集通路）
ssh afc03 "/opt/miniforge3/envs/mlk-ov/bin/python /root/rk/web/bench_stages.py"
ssh afc03 "cd /root/rk/deploy && ./affinity_test.sh /dev/video22 300"

# 起服务 + 测真实帧率
ssh afc03 "cd /root/rk/deploy && ./test_pipeline.sh 12"
```

3.10 停止 / 重启

```
ssh afc03 "pkill -f 'mipi_web[s]tream'"   # 停推流（方括号写法，避免 pkill 杀掉自己）
ssh afc03 "pkill -f 'gst-launch'"         # 停 HDMI 输出
ssh afc03 "pkill -f start_ai"             # 停 9013 独立版
ssh afc03 "reboot"                        # 重启板子
```

4. HTTP API 速查（默认 http://192.168.137.100:9012 ）

方法	路径	说明
GET	/	内置控制台页面
GET	/stream	MJPEG 流（VLC 可放）
GET	/snapshot.jpg	抓当前一帧
GET / POST	/api/params	软件画面参数（亮度/对比度/饱和度/色相/伽马/镜像/翻转/旋转/缩放/质量/帧率）

方法	路径	说明
GET	`/api/status`	后端 / 设备 / 分辨率 / 实际帧率 / 客户端数
GET / POST	`/api/fpga`	**FPGA 寄存器**：摄像头物理量、初级/高级预处理、回读快照
GET / POST	`/api/ai`	AI 状态与调参（需 `--ai` 启动）
GET	`/api/modes`	可用输出格式表（含 `impl` RTL 是否实现、 `ok` 带宽是否 ≤85%）
POST	`/api/mode`	切输出格式 `{"id":"4k25"}`（返回逐步执行 log）
GET	`/api/controls` / POST `/api/ctrl`	V4L2 硬件可控参数（自动发现）

全部带 `Access-Control-Allow-Origin: *`，所以 `file://` 打开本地 HTML 也能直接调用。

5. 板上脚本索引

脚本	用途
`deploy/rk_mipi_selftest.sh`	6 步链路自检（内核→DTS→驱动→节点→抓帧→提示 HDMI）
`deploy/test_hdmi.sh` / `hdmi_view.sh`	HDMI 出图（后者为高帧率优化版）
`web/start_webstream.sh`	一键启动网页推流（自动挑带 cv2 的解释器 + 编 C 加速库）
`deploy/restart_web.sh`	重启 web 服务并自检接口
`deploy/switch_res_mode.sh`	1080p / 4k25 / 4k30 模式切换（FPGA + media pipeline 一起改）
`deploy/fpga_cfg.py`	FPGA 寄存器读写（dump / heartbeat / mode / set / get / demo / scan）
`deploy/verify_cam_final.py`	摄像头参数 5 项验收

脚本	用途
deploy/raw_cam_dump.py / ab_raw_cam.py	原始寄存器 dump / 干净 A/B
deploy/run_verify.sh + verify_link.py	链路比特精确性验证
web/color_diag.py / color_verify.py	颜色诊断 / 逐像素比对
web/bench_stages.py / bench_new.py	逐阶段耗时 / 优化前后对比
web/cv2_probe.py	探测各 /dev/video* 能否出帧
deploy/env_check.sh	板端运行环境自检
deploy/diag_4k25.sh / diag_stream.sh	4K 不出帧 / 采集失效排查
deploy/flash_boot.sh	刷 boot 分区
ai/board/start_ai.sh	9013 独立版 AI 服务
gimbal/gimbal_keyctrl.py 、 gimbal_cmd.py	云台键盘控制 / 原始命令

6. 常见问题速查

现象	先查什么
找不到 /dev/video*	boot.img 没刷 / MIPI 互联跳线帽没装； dmesg | grep milianke
枚举为空 / 抓不到帧	FPGA bit 没烧，或 RK 没开流（ I_dsi_en 一直低）——先跑 rk_mipi_selftest.sh
提示 cv2 不可用	用错解释器：OpenCV 只在 /opt/miniforge3/envs/mlk-ov ；直接跑 start_webstream.sh 会自动选
画面发灰发绿	色彩空间：网页路径已在 mipi_webstream.py 里按 BT.709+limited 手工转换；HDMI 路径依赖驱动声明（已改 milianke_mipi.c ，需 boot.img 生效）
画面整体发白/过曝	不是颜色问题，是 IMX415 曝光/增益偏高 → 网页「摄像头」页签降低曝光/增益
帧率上不去	网页降到 960 宽 / 降到 15fps，或改 --backend gstlive ； 先确认没跑在 A53 小核上（ affinity_test.sh ）

现象	先查什么
改了 py 不生效	`restart_web.sh`（老进程还占着 `/dev/video22`）
HDMI 黑屏	`cat /sys/class/drm/card*-HDMI-A-1/status` 看 connected；用 `modetest -c` 查 connector-id
切 4K 后一帧都没有	传感器仍是 2×2 binning（`0x3020/21/22 != 0`）；tlast 永不触发，详见 `docs/IMX415_4K_MODE.md`

7. 相关文档

文档	内容
`README.md`	总部署指南（框图、硬件准备、双端步骤、上电顺序）
`rk/deploy/README.md`	RK 端刷机/部署/排错/驱动参数
`rk/web/README.md`	网页推流、采集后端对比、HTTP API、颜色根因、帧率优化
`rk/ai/README.md`	AI 模型转换与部署、危险动作规则、级联空间约束
`rk/deploy/PERFORMANCE.md`	帧率能力实测报告
`MODIFICATIONS.md`	相对官方例程的改动清单
`docs/IMX415_4K_MODE.md`	IMX415 4K 模式寄存器差表与改动清单
`docs/JPEG_ENCODER_DESIGN.md`	FPGA 硬件 JPEG 编码器设计（规划中）